

本小节内容

初始化栈-入栈-出栈实战

初始化栈-入栈-出栈实战

在上一小节我们已经讲解了栈的原理。

代码实战步骤依次是初始化栈，判断栈是否为空，压栈，获取栈顶元素，弹栈。注意 S.top 为-1 时，代表栈为空，我们每次是先对 S.top 加 1 后，再放置元素，下面直接上代码：

```
#include <stdio.h>
#include <stdlib.h>

#define MaxSize 50
typedef int ElemType;
typedef struct{
    ElemType data[MaxSize]; //数组
    int top;
}SqStack;
void InitStack(SqStack &S)
{
    S.top=-1; //代表栈为空
}

bool StackEmpty(SqStack &S)
{
    if(S.top==-1)
        return true;
    else
        return false;
}

//入栈
bool Push(SqStack &S, ElemType x)
{
    if(S.top==MaxSize-1) //数组的大小不能改变，避免访问越界
    {
        return false;
    }
    S.data[++S.top]=x;
    return true;
}
```

//出栈

```
bool Pop(SqStack &S,ElemType &x)
```

```
{
    if(-1==S.top)
        return false;
    x=S.data[S.top--];//后减减, x=S.data[S.top];S.top=S.top-1;
    return true;
}
```

//读取栈顶元素

```
bool GetTop(SqStack &S,ElemType &x)
```

```
{
    if(-1==S.top)//说明栈为空
        return false;
    x=S.data[S.top];
    return true;
}
```

//《王道 C 督学营》课程

//实现栈 可以用数组,也可以用链表,我们这里使用数组

```
int main()
```

```
{
    SqStack S;//先进后出 FILO LIFO
    bool flag;
    ElemType m;//用来存放拿出的元素
    InitStack(S);//初始化
    flag=StackEmpty(S);
    if(flag)
    {
        printf("栈是空的\n");
    }
    Push(S,3);//入栈元素 3
    Push(S,4);//入栈元素 4
    Push(S,5);
    flag=GetTop(S,m);//获取栈顶元素
    if(flag)
    {
        printf("获取栈顶元素为 %d\n",m);
    }
    flag=Pop(S,m);//弹出栈顶元素
    if(flag)
    {
        printf("弹出元素为 %d\n",m);
    }
    return 0;
}
```